Tracing Your Program's Execution

While we're having fun reflecting on all the objects and classes in our programs, let's not forget about the humble expressions that make our code actually do things. It turns out that Ruby lets us look at these expressions, too.

First, you can watch the interpreter as it executes code, using the TracePoint class. TracePoint is used to execute a proc while adding all sorts of juicy debugging information whenever a new source line is executed, methods are called, objects are created, and so on.

Here's a bit of what TracePoint can do (run this in your own irb window, but be aware that it will produce a *lot* of output):

```
class Test
  def test
    a = 1
  end
end

TracePoint.trace(:line) do |trace_point|
  p trace_point
end

t = Test.new
t.test
```

The method trace_var lets you add a hook to a global variable; whenever an assignment is made to the global, your proc is invoked.

How Did We Get Here?

That's a fair question…one we ask ourselves regularly. Mental lapses aside, in Ruby you can find out "how you got there" using the method caller, which returns an array of strings representing the current call stack:

```
def cat_a
  puts caller[0..2]
end

def cat_b
  cat_a
end

def cat_c
  cat_b
end

cat_c
```

produces:

```
prog.rb:6:in `cat_b'
prog.rb:10:in `cat_c'
prog.rb:13:in `<main>'
```

In this case, we're limiting printing the first three elements of the call stack because beyond that, we get into the depths of irb, which is unlikely to be useful for debugging purposes.

Ruby also provides the method __callee__, which returns the name of the current method.